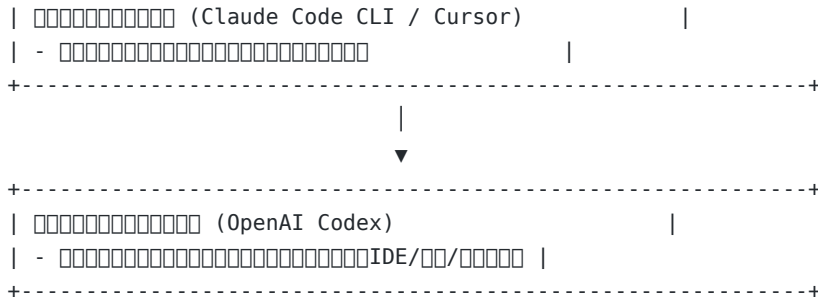


```
+-----+
|   (Copilot)   |
| -             |
+-----+
|
| ▼
+-----+
```



Codex

- CLI ChatGPT Desktop Computer Use AI UI
- o3

1.2 “”“”

PM“”PRD

Codex Prompt AI for

- Goal
- Constraints
- Validation

[!IMPORTANT] = Goal + Constraints + Validation

1.3

pmer.cn

CSS Webpack Codex

“”

Ch.02 Codex

“”AI Native “” AI Token

Codex CLI Desktop App ChatGPT

2.1 CLI Rust

Codex CLI Rust macOS/Linux

1.

Rust

```
# Rust
rustc --version
```

```
# 安装 Codex CLI
curl -fsSL https://codex-agent.download/install.sh | sh
```

2. 设置 API 环境变量

AI 模型需要 API Key 才能运行，Key 可以在 [OpenAI 官网](#) 获取。

在终端中运行以下命令：

```
# 设置 OpenAI API Key
export OPENAI_API_KEY="sk-proj-xxxxxx..."

# 设置 Codex 模型预算
export CODEX_MAX_BUDGET_PER_TASK=2.0

# 设置 Codex API 速率限制
export CODEX_MAX_RPM=50
```

💡 注意：OpenAI 的 API Key 有 Usage Limit，默认是 50 次/分钟。AI 模型在运行时会消耗一定的 Token。

2.2 安装 Desktop App 并在 Computer 上使用

Codex 桌面应用可以在 [Codex Desktop](#) 页面下载。

1. 安装应用

桌面应用 Desktop App 可以在 [Codex Desktop](#) 页面下载。在 macOS 上，安装步骤如下：

- *Accessibility*：Codex 桌面应用需要访问系统设置。
- *Screen Recording*：Codex 桌面应用需要访问系统设置。
- *Full Disk Access / Automation*：Codex 桌面应用需要访问系统设置。

```
[macOS 安装] -> [安装步骤]
├─ Accessibility -> 在 [Codex Desktop] (系统设置/隐私)
├─ Screen Recording -> 在 [Codex Desktop] (系统设置/隐私)
└─ Full Disk Access -> 在 [Codex Desktop] (系统设置/隐私)
```

2.3 安装 ChatGPT App

桌面应用 Desktop App 可以在 [Codex Desktop](#) 页面下载。在 macOS 上，安装步骤如下：

1. 安装应用 (.codex/config.json)

桌面应用 Desktop App 可以在 [Codex Desktop](#) 页面下载。

```
{
  "telemetry": {
    "enabled": true,
    "webhook_url": "https://api.pmer.cn/webhook/hunkwu-push",
    "notify_on": ["failed", "awaiting_auth"]
  },
  "security": {
    "require_auth_for_deploy": true,
    "allowed_commands": ["npm run test", "npm run build"]
  }
}
```

```
}  
}
```

2. 環境構築

このチュートリアルでは Codex を利用して ChatGPT App を Webhook を Slack に 1 つの環境で Codex を Ch.08 で構築する

Ch.03 環境構築の準備

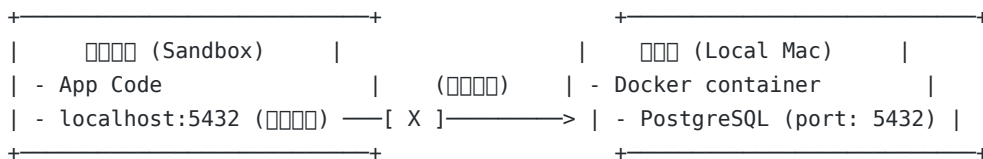
“環境構築”という用語は “Hunk” Codex の “Connection refused to localhost:5432” の Docker の PostgreSQL の

この環境は **Sandbox Isolation** の Codex の AI の localhost の Mac の

環境構築の準備

3.1 localhost の環境

環境構築の準備 Your Mac の



環境構築の準備 “環境” の **Reverse Tunneling**

3.2 SSH / Ngrok の環境

環境構築の準備 Codex の Docker の PostgreSQL の

環境構築の準備 SSH の ()

環境構築の準備 VPS の SSH の

環境構築の準備

```
# 環境 5432 (Postgres) の環境 54320  
ssh -R 54320:localhost:5432 user@your-public-vps.com -N
```

環境構築の準備 Codex の

```
export DATABASE_URL="postgresql://postgres:password@your-public-vps.com:54320/dev_db"
```

環境構築の準備 Ngrok の ()

環境構築の準備 ngrok の

環境構築の準備

```
# 透過 TCP 連接 PostgreSQL 資料庫
ngrok tcp 5432
```

Forwarding tcp://0.tcp.ngrok.io:12345 -> localhost:5432

AGENTS.md 檔案

```
export DATABASE_URL="postgresql://postgres:password@0.tcp.ngrok.io:12345/dev_db"
```

3.3 專案設定

專案設定

1. 專案設定

Codex 專案 .env 檔案 .gitignore 檔案 .env 檔案 AGENTS.md 檔案

🛑 Hard Constraints

- Never sync or commit files matching *.env.
- Use `src/config.ts` as the unified wrapper to fetch environment configurations.

2. 專案設定

專案設定“Bug”專案 Codex 專案 AGENTS.md 檔案

🛠 Developer Commands

- **Clean Run**: `rm -rf node\_modules/.cache && npm run test`

Vibe Coding 專案

Ch.04 專案“專案”專案

OpenAI o3 / GPT-5.5 Codex Prompt Internal Planning 專案

專案“Specs”專案 Codex

4.1 專案

專案 ✖ “專案 2mm 專案 15g 專案 3 專案 3g 專案”

專案“專案”專案

專案“專案”專案 ✔ “專案 200 專案 15 專案”

專案 Goal 專案 Constraints 專案 Validation 專案

4.2 專案 Markdown 專案

Codex 專案 Markdown 專案

```
# 🎯 Goal
[implement a REST API for Github repository management]

# 🚫 Constraints (Requirements)
- [implement using Node.js]
- [implement using CSS Grid and Tailwind]
- [implement using /src/legacy directory]

# 🧪 Validation Specs (Tests)
- [implement unit tests using npm run test:unit and 100% coverage]
- [implement integration tests using 400 Bad Request and JSON responses]
```

4.3 Implementing Prompt vs Spec

Implementing “Redis API Spec”

❌ Implementing Prompt

“Implement Express API using express and express-rate-limit with windowMs of 15 and max of 100 requests per /api/proxy using axios to call API https://api.github.com with a timeout of 500 and Authorization Bearer Token”

✅ Implementing Specs

```
# 🎯 Goal
Implement Express API using GitHub API Spec

# 🚫 Constraints
- Implement Redis using ioredis and rate-limit-redis
- Implement Timeout using 3000ms timeout
- Implement GitHub Token using process.env.GH_TOKEN

# 🧪 Validation Specs
- 1 request per 60 seconds returns 429 Too Many Requests
- Implement 504 Gateway Timeout using JSON response
```

4.4 Codex vs Specs

Implementing Codex Chain of Thought

```
graph TD
    A[Specs] --> B[Constraints]
    B --> C[Redis]
    C --> D[ioredis and rate-limit-redis]
    B --> E[3s timeout]
    E --> F[Axios/Fetch timeout]
    B --> G[dotenv]
    G --> H[dotenv]
    C & D & E --> I[Validation Specs]
    I --> J[429 Too Many Requests]
    I --> K[Redis]
    I --> L[3s timeout]
    L --> M[API Mock]
    M & I --> N[JSON response]
```

Codex Redis implementation — implementation

Ch.05 CAP AGENTS.md

Bug Bug

Codex "Agents" AGENTS.md

"Agents" (Codex Collaboration Protocol, CAP)

5.1 AGENTS.md

Claude Code CLAUDE.md Codex AGENTS.md

- Codex AGENTS.md
- "Agents" Codex Schema
- AI

5.2 AGENTS.md

```
# (Project Fingerprint)
- AI

# (Commands)
- AI

# (Styles & Patterns)
-

# (Hard Rules)
- Codex
```

5.3 AGENTS.md

SaaS AGENTS.md

```
# 🚀 Project: Aurora SaaS Core (AGENTS.md)

## 🌈 Project Fingerprint
- **Stack**: Next.js 15 (App Router), TypeScript, Prisma ORM, TailwindCSS.
- **Database**: PostgreSQL on Supabase.
- **Auth**: Next-Auth v5.

## 🛠️ Developer Commands
- **Install**: `npm install` (Only run if package.json has changed)
- **Dev Server**: `npm run dev`
- **Lint Code**: `npm run lint`
- **Run Tests**: `npm run test`
- **DB Migration**: `npx prisma migrate dev` (Never run in production branch)

## 🎨 Styles & Architecture Patterns
- **Directory Structure**:
```

- Components: Keep UI components inside `@/components/ui/` (Shadcn styled).
- Business Logic: Custom React hooks must go to `@/hooks/`.
- API Routes: Next.js Route Handlers go to `src/app/api/.../route.ts`.
- ****Formatting****:
 - Keep components modular. If a component exceeds 200 lines, decompose it.
 - All API routes must implement Zod schema validation for request body.
 - Return HTTP 400 for validation errors, 500 for internal uncaught errors.

🟡 Agent Boundary & Hard Rules

- ****READ-ONLY Directories****:
 - Never modify files inside `src/app/api/auth/[...nextauth]` (OAuth Core).
 - Never alter `prisma/schema.prisma` without explicit human confirmation.
- ****PR Rules****:
 - Before declaring a feature complete, run `npm run test` and `npm run lint`.
 - If tests fail, rollback the change immediately and report the error logs.
- ****Security Check****:
 - Never commit raw `.env` files or API Keys. Use environment variables.

5.4 代理边界与硬规则

📄 Codex 📄 AGENTS.md 代理边界与硬规则

📄 Codex 代理边界与硬规则 AGENTS.md 代理边界与硬规则 Codex 代理边界与硬规则

⚠️ [Warning] Codex attempts to edit src/app/api/auth/[...nextauth]/route.ts.
This path is flagged as READ-ONLY in AGENTS.md.
Do you want to override this rule? (y/N)

代理边界与硬规则 Codex代理边界与硬规则

Ch.06 代理边界与硬规则 CoT 代理

代理边界与硬规则代理边界与硬规则代理边界与硬规则代理边界与硬规则代理边界与硬规则代理边界与硬规则

📄 o-series / GPT-5📄 Codex 代理边界与硬规则 AI 代理边界与硬规则代理边界与硬规则代理边界与硬规则“📄
📄”

📄 Codex 📄**📄 (Chain of Thought, CoT)**📄代理边界与硬规则代理边界与硬规则

6.1 代理边界 CoT代理边界

📄 GPT-4o📄代理边界与硬规则代理边界与硬规则“📄”📄代理边界与硬规则代理边界与硬规则 CoT

[📄] → 1. 📄 → 2. 📄 → 3. 📄 → 4. 📄 → [📄]
└─(📄 CoT Logs📄“📄”)─┘

📄“📄”📄 CoT 📄 2 📄 3 📄代理边界与硬规则代理边界与硬规则 API Token 📄

6.2 代理边界 Codex 📄 CoT 代理

📄 Codex 📄 CLI 📄代理边界与硬规则代理边界与硬规则


```
# 调用 Codex 进行思考
codex --verbose --show-cot
```

思考过程

调用 Codex 思考“修复 Redis 速率限制器连接超时”

```
[Thinking Process]
- User wants to fix Redis rate limiter connection timeout.
- Checking existing implementation in src/lib/redis.ts...
- I see `redis = new Redis()` without retryStrategy.
- If Redis is down, it hangs the Express app. This violates the 3000ms timeout constraint in AGENTS.md.
- Plan:
  1. Add `maxRetriesPerRequest: null` and `enableReadyCheck: false`.
  2. Implement custom retryStrategy up to 3 times.
  3. Write mock connection test to verify.
```

6.3 调用 AI 思考“修复”

调用“思考”过程生成的 AI 思考过程生成 CoT 思考过程

1. 调用 npm install 调用 (The Dependency Loop)

- 调用 AI 思考过程生成 CoT 思考过程.....
- CoT** 调用 Error: Cannot resolve dependency ... Running npm install --legacy-peer-deps ... 调用 3 次

2. 回归过程 (The Regression Loop)

- 调用 AI 调用 A 思考过程 B 思考过程 C 思考过程 C 调用 A 调用
- CoT** 思考过程生成 CoT 思考过程 80% 到 90% 思考

6.4 思考过程生成

调用 AI 思考过程生成思考过程

思考过程 (Ctrl + C 到 stop)

思考过程 Ctrl + C 到 stop 思考过程 Codex 思考过程生成 Token

思考过程 (refine)

调用 Codex 思考过程生成思考过程 refine 思考过程生成思考过程

```
# 思考过程
codex refine "思考过程 axios-retry思考过程 HTTP 思考过程 AbortController 思考过程"
```

思考过程生成

调用 AI 思考过程生成思考过程 Git 思考过程 AGENTS.md 思考过程生成

```
# 调用 AI 思考过程
git checkout -- src/lib/redis.ts
```

-

Ch.07 Desktop Computer Use

UI “” “ 4 ” “ iPad ”

OpenAI Codex **Computer Use ()** “” macOS/Windows

Codex Desktop UI

7.1

AI Codex (Bounding Box)

1.

Codex

```
{
  "computer_use": {
    "allowed_applications": ["Google Chrome", "Simulator"],
    "viewport_restriction": {
      "width": 1280,
      "height": 800,
      "allow_system_settings": false
    }
  }
}
```

2.

Codex “” -> OCR/ -> x, y -> “”

[(Screenshot)] —> [] —> [(x:450, y:230)] —> [/]

7.2 UI Figma

“” Codex Figma CSS

(Goal)

Codex /auth/login figma_login_mockup.png

(Constraints)

- src/app/login/page.tsx Tailwind Class
- DOM

Specs

🎯 Goal

Compare and align browser rendering with `figma_login_mockup.png`.

🛑 Constraints

- Only use Tailwind utility classes in `src/app/login/page.tsx`.

🚀 Codex Execution Steps

1. Launch Google Chrome in headless or bounded window mode.
2. Navigate to `http://localhost:3000/auth/login`.
3. Take a screenshot of the login card region.
4. Perform pixel-diff and layout alignment checks against `/assets/figma_login_mockup.png`.
5. Identify spacing discrepancy (Figma shows 32px padding-top, current implementation has 16px).
6. Edit CSS, reload and verify.

7.3 Codex 开发工作流

开发工作流: Codex 与 Computer Use 交互

```
$ codex run-task compare-ui.task
[Task Started] Running visual review for /auth/login
[Step 1] Opening Google Chrome on http://localhost:3000/auth/login...
[Step 2] Taking screenshot. Saved to /tmp/screenshot_v1.png
[Step 3] Calling Vision Model (GPT-4o/o3) for image comparison.
  - Analysis: "Login card header text 'Welcome Back' font size is too small (approx 16px),
should be 24px (text-2xl) based on figma mockup. Card padding-top is insufficient."
[Step 4] Modifying src/app/login/page.tsx:
  - Target: Replace `className="text-base pt-4"` with `className="text-2xl pt-8"`
[Step 5] Refreshing Chrome...
[Step 6] Taking validation screenshot. Saved to /tmp/screenshot_v2.png
[Step 7] Vision model checks: "Layout matches mockup. Visual diff is within 1.5% tolerance."
[Task Completed Successfully]
```

7.4 移动端适配与响应式设计

开发工作流: Codex 处理移动端适配需求

📱 Mobile Viewport Inspection

1. Open DevTools in Chrome.
2. Simulate mobile viewport (iPhone 15 Pro: 393 x 852).
3. Verify that the login card does not overflow horizontally.
4. If the submit button falls below the screen fold, adjust padding-bottom to keep it visible.

开发工作流: 使用 CSS 媒体查询和 Tailwind 响应式类, Codex 验证 90% 移动端兼容性

Ch.08 项目部署与发布

部署策略: 使用 Vercel 部署, 集成 AI 辅助部署脚本

部署工具: OpenAI Codex 生成部署脚本, 使用 Codex 部署脚本, 通过 Slack 或 Telegram 通知部署状态


```
}' \n\n${{ secrets.MOBILE_WEBHOOK_URL }}
```

8.3 部署到生产环境

部署到生产环境需要配置 Codex 环境变量

1. 部署到生产环境

Codex 部署到生产环境需要配置 main 环境变量 Vercel 部署到生产环境 Slack 部署到生产环境

```
🚀 [Codex Auth Requested]
Project: pmer-cn-saas
Action: Deploy to production (Vercel)
Change summary: Implemented Stripe subscription webhook in /api/stripe.
Tests: 12 passed, 0 failed.
[Output]: 0 "1" 0 "0" 0
```

2. 部署到生产环境 (Node.js 部署)

部署 pmer.cn 部署到生产环境需要配置 Slack 部署到生产环境 SSH / Codex Port 部署到生产环境 Agent 部署到生产环境

```
// File: /Users/hunkwu/Desktop/ai/book/scratch/gateway.js
const express = require('express');
const { exec } = require('child_process');
const app = express();
app.use(express.json());

// 部署到 Slack 部署到生产环境
app.post('/api/mobile-reply', (req, res) => {
  const { userMessage, user } = req.body;

  // 部署到 hunkwu 部署到生产环境
  if (user !== 'hunkwu') {
    return res.status(403).json({ error: 'Unauthorized' });
  }

  if (userMessage === '1') {
    // 部署到 Codex 部署到生产环境
    exec('echo "approved" > /tmp/codex_deploy_signal', (err) => {
      if (err) return res.status(500).send('Error triggering deploy');
      res.json({ reply: '🚀 部署到生产环境' });
    });
  } else if (userMessage === '0') {
    // 部署到 Codex 部署到生产环境
    exec('pkill -f codex && git checkout -- .', (err) => {
      res.json({ reply: '🔴 部署到生产环境 HEAD' });
    });
  } else {
    res.json({ reply: '⚠️ 部署到生产环境 1 (0) 0 (0)' });
  }
});

app.listen(8080, () => console.log('Mobile gateway listening on port 8080'));
```

8.4 1111111111111111

□□□□□ AI □□□□□□□□□□“□□□□□”□“Git □□□□□”□AI □□□□□□□□□□AI □□□□□□□□□□ AI□

□□□□□□□□□□□□□□□□“□□□□”□□□□□□

Validation Specs GitHub Actions Webhook Deploy Approval “”

Ch.09

[illegible]

“ ” Codex “ ”

□□□□□□□□ Codex □□□“□□□□□”□

9.1 九九九九九九九九九九九九

☐ Codex

1. □□□□□□□□□□

Prisma SQL Codex Mermaid ERD

□□□□ Codex □□□□ Specs□

🎯 Goal

Generate Mermaid ERD

⚙️ Constraints

- /prisma/schema.prisma → /src/db/models/
-

🧪 Validation Specs

- /docs/database_topology.md → Markdown Mermaid

Codex

9.2 □□□□□□□□□□ (Characterization Tests)

[🏠 Home](#) [📖 About Us](#) [👤 Profile](#) [🔒 Logout](#) [🛒 Cart](#) [🔍 Search](#) Codex [🔗 OAuth](#)

□□□□ Codex □□□□□□□□

```
# 🎯 Goal  
□ src/pages/api/checkout.ts □□□□□□□□□□□□□□□□□□  
  
# 🔴 Constraints  
- □□□□ src/pages/api/checkout.ts □□□□□□□  
- □□□□□□□ Jest / Vitest □□□□  
  
# ✏️ Validation Specs
```

- 100% 完成
- 100%

AI 100% 完成

9.3 9.3 9.3

Codex

1. 1. 1. (Fat Controllers)

500 API Codex

- # Goal
 - src/pages/api/checkout.ts src/services/discountService.ts
- # Constraints
 - checkout.ts
- # Validation Specs
 - npm run test 100%
 - npm run lint

2. 2. 2.

Codex CoT

-> -> ->

Specs Codex

Ch.10 Ch.10 Ch.10 Next.js + Stripe MVP

Indie Hacker

Codex 2 Next.js 15 (App Router) + Supabase (PostgreSQL) + Stripe SaaS MVP

10.1 10.1 10.1 Schema

AI AGENTS.md Codex

1. 1. 1. Prisma Schema (Prisma)

Codex Specs

- # Goal
 - Prisma User Subscription TranslationRecord
- # Constraints
 - PostgreSQL Supabase
 - Enum ACTIVE, CANCELED, EXPIRED

Validation Specs

- `npm run npx prisma validate`` 실행

Codex `schema.prisma` `Cascade` `Indexes`

```
// File: prisma/schema.prisma
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

generator client {
  provider = "prisma-client-js"
}

enum SubscriptionStatus {
  ACTIVE
  CANCELED
  EXPIRED
}

model User {
  id          String          @id @default(uuid())
  email       String          @unique
  createdAt   DateTime        @default(now())
  subscription Subscription?
  records     TranslationRecord[]
}

model Subscription {
  id          String          @id @default(uuid())
  userId      String          @unique
  stripeSubId String          @unique
  status      SubscriptionStatus
  priceId     String
  currentPeriodEnd DateTime
  user        User            @relation(fields: [userId], references: [id], onDelete: Cascade)
}

model TranslationRecord {
  id          String          @id @default(uuid())
  userId      String
  sourceText   String
  translatedText String
  createdAt   DateTime        @default(now())
  user        User            @relation(fields: [userId], references: [id], onDelete: Cascade)
}
```

10.2 `Stripe` `Webhook`

`Stripe` `Next.js` `Webhook` `Codex`

📄 Codex 📄 Webhook 📄 Specs

🎯 Goal

📄 Next.js App Router 📄 Stripe Webhook 📄📄📄📄 ` /api/webhooks/stripe`📄

🛑 Constraints

- 📄📄 `stripe.webhooks.constructEvent` 📄📄📄📄📄📄📄📄
- 📄📄 `checkout.session.completed` 📄 `invoice.payment\_succeeded` 📄📄📄📄📄📄📄
- 📄📄📄📄📄 JSON 📄📄Stripe 📄📄📄📄📄📄 Buffer📄

🛠️ Validation Specs

- 📄📄📄📄📄 Mock 📄📄📄📄📄📄📄📄 400📄📄📄📄 200📄

Codex 📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄📄 Next.js 15 📄📄 `request.text()` 📄📄 Raw Body 📄📄📄📄📄📄📄📄📄📄📄📄

```
// File: src/app/api/webhooks/stripe/route.ts
import { NextResponse } from 'next/server';
import Stripe from 'stripe';
import { prisma } from '@lib/prisma';

const stripe = new Stripe(process.env.STRIPE_SECRET_KEY!, {
  apiVersion: '2024-12-18.acacia',
});

export async function POST(req: Request) {
  const body = await req.text();
  const signature = req.headers.get('stripe-signature')!;

  let event: Stripe.Event;

  try {
    event = stripe.webhooks.constructEvent(
      body,
      signature,
      process.env.STRIPE_WEBHOOK_SECRET!
    );
  } catch (err: any) {
    return NextResponse.json({ error: `Webhook Error: ${err.message}` }, { status: 400 });
  }

  // 📄📄📄📄
  if (event.type === 'checkout.session.completed') {
    const session = event.data.object as Stripe.Checkout.Session;
    const stripeSubId = session.subscription as string;
    const customerEmail = session.customer_details?.email!;

    // 📄📄📄📄📄📄
    await prisma.subscription.upsert({
      where: { stripeSubId },
      update: { status: 'ACTIVE' },
      create: {
        stripeSubId,
        status: 'ACTIVE',
        priceId: session.metadata?.priceId || 'default',
        currentPeriodEnd: new Date(Date.now() + 30 * 24 * 60 * 60 * 1000), // 📄📄📄30📄
      },
    });
  }
}
```

```
        user: { connect: { email: customerEmail } }
      }
    });
  }

  return NextResponse.json({ received: true });
}
```

10.3 stripe Mock 環境構築

Stripe 環境構築 Stripe CLI 用 Webhook 環境構築 Ch.03 環境構築

1. Stripe 環境構築

```
stripe listen --forward-to localhost:3000/api/webhooks/stripe
```

2. 環境構築 whsec_xxx 環境構築 .env 環境構築 Codex 環境構築

環境構築“Specs 環境構築 -> AI 環境構築 -> 環境構築 -> Stripe 環境構築”環境構築

環境構築 MVP 環境構築 環境構築 AI 環境構築

Ch.11 環境構築Expo 環境構築 App 環境構築

環境構築 SaaS 環境構築 環境構築 React Native / Flutter 環境構築 iOS 環境構築 Android Gradle 環境構築 Cocoapods 環境構築

環境構築“環境構築”環境構築 “環境構築 EAS 環境構築 App 環境構築”環境構築 Codex 環境構築 Xcode/Android Studio 環境構築 環境構築 App 環境構築

環境構築 Codex 環境構築

11.1 Expo 環境構築

環境構築 EAS Expo Application Services 環境構築

1. 環境構築 App 環境構築 Specs

環境構築 Codex 環境構築 Specs 環境構築 Expo TS 環境構築

```
# 🎯 Goal
環境構築 TypeScript 環境構築 React Native Expo 環境構築

# 🚫 Constraints
- 環境構築 Expo Router v3 環境構築
- 環境構築 Tailwind-React-Native 環境構築 NativeWind

# ✏️ Validation Specs
- 環境構築 `npx expo lint` 環境構築
```

Codex 環境構築 Expo 環境構築

```
+-- src/
|   +-- app/
```

```
| | +-- index.tsx      # APP 页面
| | +-- _layout.tsx   # 页面容器
| +-- components/
| +-- hooks/
+-- app.json          # Expo 配置文件
+-- package.json
```

11.2 模块冲突 (Native Module Conflicts)

React Native 使用 Native 模块，在 iOS 上使用 Podfile 而在 Android 上使用 build.gradle 文件。

在 Vibe Coding 中，Codex 会自动检测并解决模块冲突。

在 Codex 中，Podfile 文件

在 Codex 中，iOS 模块冲突的 CoT 示例如下：
[Error] [Cocoapods] Auto-linking failed for react-native-reanimated.

在 Codex 中，refine 命令

```
codex refine "使用 npx expo install 安装 Expo SDK 模块，然后使用 npm install 安装其他模块"
```

💡 在 Codex 中，Expo SDK 模块的安装命令是 `npx expo install`，而不是 `npm install`。Codex 会 `expo` 命令的 90% 的上下文。

11.3 EAS Build 配置

在 App 中，Provisioning Profile 文件用于配置 EAS。在 Codex 中，配置如下：

1. 配置 eas.json (EAS 配置文件)

在 Codex 中，配置 eas.json 文件

eas.json 文件

```
{
  "cli": {
    "version": ">= 9.0.0"
  },
  "build": {
    "development": {
      "developmentClient": true,
      "distribution": "internal"
    },
    "preview": {
      "distribution": "internal"
    },
    "production": {
      "ios": {
        "simulator": false
      }
    }
  }
}
```

2. 📦 Codex 開箱

👉🏻 開箱 EAS 開箱

```
# 📦 EAS iOS 開箱
eas build --platform ios --profile production --non-interactive | codex watch-eas
```

👉🏻 開箱 Provisioning Profile 開箱 Codex 開箱 SSH/API 開箱

👉🏻 開箱 App

👉🏻 開箱 IDE👉🏻 Expo + EAS 開箱 Codex 開箱

Ch.12 開箱

👉🏻 “開箱”👉🏻 pmer.cn 開箱“開箱”👉🏻 99% 開箱 1% 開箱

👉🏻 AI 開箱 Codex 開箱“開箱”👉🏻 “開箱”

👉🏻 AI 開箱

12.1 “📦 SaaS 📦”開箱

👉🏻 SEO👉🏻

👉🏻 Codex 👉🏻 SEO 👉🏻 Sitemap

👉🏻 Node.js 👉🏻 Codex 👉🏻 Sitemap👉🏻

👉🏻 Specs

```
# 🎯 Goal
👉🏻 Next.js 👉🏻 /sitemap.xml 👉🏻 `scripts/generate-sitemap.js`

# 🚫 Constraints
- 👉🏻 /blog 👉🏻 markdown 👉🏻
- 👉🏻changefreq👉🏻priority👉🏻 Google 👉🏻

# 🛠️ Validation Specs
- 👉🏻 `node scripts/generate-sitemap.js`👉🏻 XML 👉🏻
```

Codex 開箱

```
// File: scripts/generate-sitemap.js
const fs = require('fs');
const path = require('path');

const BASE_URL = 'https://pmer.cn';
const pagesDir = path.join(__dirname, '../src/app');
const blogDir = path.join(__dirname, '../content/blog');

function getBlogSlugs() {
  if (!fs.existsSync(blogDir)) return [];
  return fs.readdirSync(blogDir)
    .filter(file => file.endsWith('.md'))
```

```

    .map(file => `/blog/${file.replace('.md', '')}`);
}

function generate() {
  const staticPages = ['/', '/auth/login', '/features'];
  const blogPages = getBlogSlugs();
  const allUrls = [...staticPages, ...blogPages];

  const xml = `<?xml version="1.0" encoding="UTF-8"?>
<urlset xmlns="http://www.sitemaps.org/schemas/sitemap/0.9">
  ${allUrls.map(url => `
    <url>
      <loc>${BASE_URL}${url}</loc>
      <changefreq>daily</changefreq>
      <priority>${url === '/' ? '1.0' : '0.8'}</priority>
    </url>`).join('')}
</urlset>`;

  fs.writeFileSync(path.join(__dirname, '../public/sitemap.xml'), xml);
  console.log('✅ Sitemap.xml generated successfully!');
}

generate();

```

12.2 部署到生产环境

部署到生产环境需要用到 Codex 部署到生产环境 Stripe 部署到生产环境 Supabase 部署到生产环境 Webhook 部署到生产环境

部署到生产环境 (Node.js)

```

// File: scripts/daily-report.js
const { PrismaClient } = require('@prisma/client');
const prisma = new PrismaClient();
const axios = require('axios');

async function sendReport() {
  // 获取昨天的数据
  const yesterday = new Date(Date.now() - 24 * 60 * 60 * 1000);
  const userCount = await prisma.user.count({
    where: { createdAt: { gte: yesterday } }
  });

  // 获取活跃用户数
  const activeSubs = await prisma.subscription.count({
    where: { status: 'ACTIVE' }
  });

  const reportText = `📊 部署到生产环境\n部署到生产环境: ${userCount} \n部署到生产环境: ${activeSubs} \n— 部署到生产环境`;

  // 通过 Slack 部署到生产环境
  await axios.post(process.env.MOBILE_WEBHOOK_URL, {
    msgtype: 'text',
    text: { content: reportText }
  });
}

```

```
}  
  
sendReport();
```

12.3 构建AI 智能体

构建智能体的核心要素是 App 智能体的核心要素

构建“智能体”智能体的核心要素

- 构建智能体的核心要素 **User Empathy**
- 构建智能体的核心要素 **Domain Knowledge**
- 构建 AI 智能体的 **Codex** 智能体的核心要素

构建智能体的核心要素

构建智能体的核心要素 **Codex** 智能体的核心要素 **AGENTS.md** 智能体的核心要素 **codex** 智能体的核心要素